Lecture-6

Software Testing

- Is often referred to as verification and validation (V&V).

- Verification
  - Activities ensure that implementation has a specific function
  - *Are we building the product right?*

- Validation
  - Activities ensure that software traceable to requirements
  - *Are we building the right product?*

Prin testare, vedem daca sistemul face ce trebuie
Descompunem testarea in:
1. Verificare (Vedem daca implementarea functioneaza corect -> anomalii de functionare + tine de functionarea fiecarei bucati)
2. Validare (Se refera la cerinte, daca a fost implementat ce trebuie si daca functioneaza ~ dupa verificare)

What Testing Shows?

- The goal of defect testing is to discover defects in programs
- A successful defect test is a test which causes a program to behave in an anomalous way

Testing can only show the presence of errors, not their absence

E.W.Dijkstra, 1972

```
Testarea imi arata doar prezenta erorilor -> nu imi arata si absenta lor
(daca de ex imi pica un test -> am eroare, dar daca nu imi pica niciun test,
tot pot sa am eroare)
Testarea e o activitate pesimista → nu iti da nicio garantie, pt ca nr de
cazuri care trebuiesc testate e ft mare, infinit de mare
```

Un test bun e acela care esueaza

Testing strategy

- We begin by 'testing-in-the-small' and move toward 'testing-in-the-large'

- Conventional Software
  - The module (component) is our initial focus
  - Integration of modules follows

- OO Software
  - our focus when "testing in the small" changes from an individual module (the conventional view) to an OO class that encompasses attributes and operations and implies communication and collaboration

Intai testam unit-urile (modulele) si apoi sistemul

```
Verificare formala -> demonstrezi matematic ca un program este corect -> e
diferit de testare
Verificarea formala vine cu o investitie -> de calcul si de specificare a
cerintelor (trebuie ca specificarea cerintelor sa fie formala -> un cost
suplimentar) -> implica costuri mari -> daca se merita, platesti, daca nu ->
test
Testarea e imperfecta !!
Avem un nr maxim de incercari de a gasi cele mai rele probleme -> cine
reuseste sa gaseasca cele mai multe erori cu resursele pe care le are ->
succesul erorii e dat de nr de erori pe care le poate gasi si nu de
garantarea absentei erorilor
Strategie de testare : de la mic la mare (pornesc de la cel mai mic modul pe
care il pot testa)
Avem testare unitara (testarea fiecarui modul in parte : o functie, o clasa
etc.) si testare de integrare (compun intregul, testez ,integrez , repet
etc. Apar si aici cerintele nefunctionale, ele sunt valabile in general pt
un sistem SW
Criterii nefunctionale : perfomanta, securitatea, mentenanta -> le-am vazut
mai sus
```

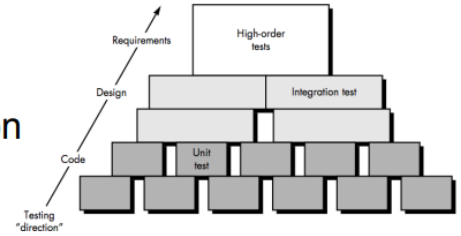
When is testing complete?

- There is no definitive answer to this question
- Every time a user executes the software, the program is being tested
- Sadly, testing usually stops when a project is running out of time, money, or both
- One approach is to divide the test results into various severity levels
  - Then consider testing to be complete when certain levels of errors no longer occur or have been repaired or eliminated

```
Niciodata testarea nu va fi completa -> de fiecare data cand un user executa
sistemul SW -> programul se testeaza -> trebuie sa divizam rezultatele de
test in nivele de severitate -> consideram ca avem testarea “completa” cand
nu mai avem nivele de erori
```

Four testing steps

Four Testing Steps



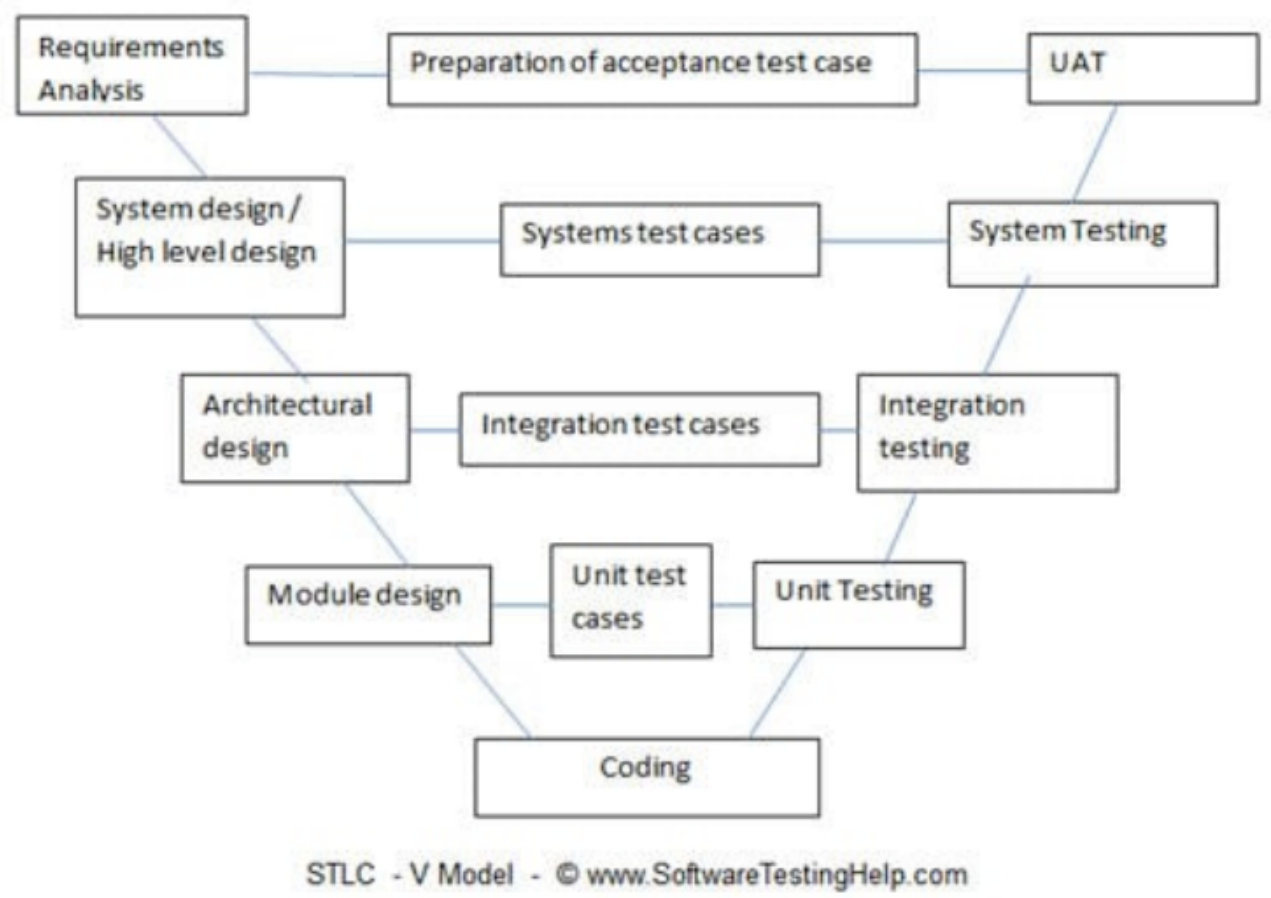
- **Step1**: Focus on each component in isolation
  - unit testing
  - heavy use of white-box testing techniques
    - exercise specific paths in the control structure for complete coverage
- **Step2**: Assemble components (integration)
  - mainly uses black-box testing techniques (may also uses white-box tests)
    - reason in terms of inputs and expected outputs
- **Step3**: Conduct higher-order tests
  - validation criteria must be set (related to requirements)
    - related to behavior and performance
  - uses exclusively black-box techniques
- **Step4**: Combine with other systems
  - goes beyond SWE ... more systems engineering

o 1 : testam fiecare componenta in izolare : testare unitara (nu inseamna ca nu are bug-uri-> folosim white box testing techniques -> e usor sa urmarim si sa scriem cod de test pt fiecare modul separat)
o 2 : testare de integrare -> merge gradual -> am un singur sistem (folosim in mare black box testing, ai mai multe module e mai greu sa scrii cod de testare bazat pe cod)
o 3 : teste de nivel inalt (testare de performanta, de cerinta, de utilizare, de integrare cu alte

sisteme pe care nu le-am implementat noi etc.)
o 4 : testarea cu alte sisteme pe care nu le-am implementat noi

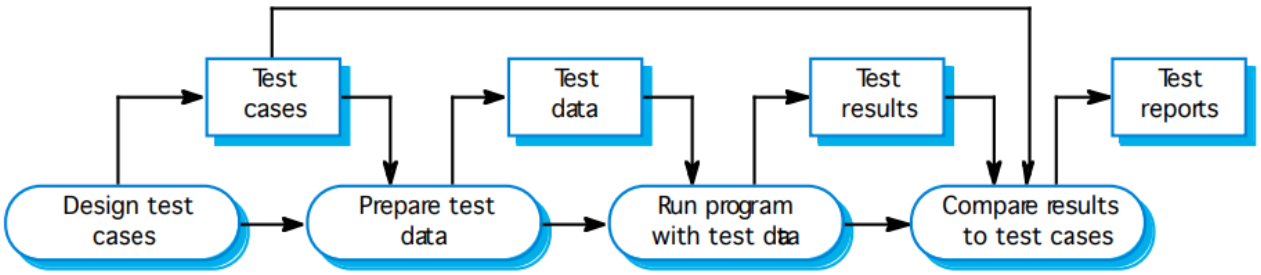
```
Avem 2 tipuri de testare : white box (vedem codul) si black box(nu vedem
codul, ci doar inputul si outputul pe care il promite)
La nivelul cel mai de jos ma astept sa am codul -> ma uit la cod si scriu
teste in functie de ce inteleg din codul ala -> s.n. testare unitara si se
face prin testare white box(vad cod)
Trec la nivelul urmator -> integrarea : am 1000 de unitati, fiecare trecuta
de testarea unitara -> le privesc ca pe o cutie neagra, fiecare cu intrari
si iesiri -> testare black box (vad specificatii) -> incep sa testez
bazandu-ma strict pe interfete -> indiferent de nivelul la care fac testarea
-> tot ce e peste testarea unitara s. n. testare black box
Se poate face bineinteles si testare de integrare unde sa am si testare
white box si black box -> pe masura ce creste nivelul de integrare, tot mai
putin faci white boxing si tot mai mult black boxing -> testez pe baza de
in/out
Trebuie sa generez date de test care sa genereze erori!!!!
```

V-Model of Software Testing

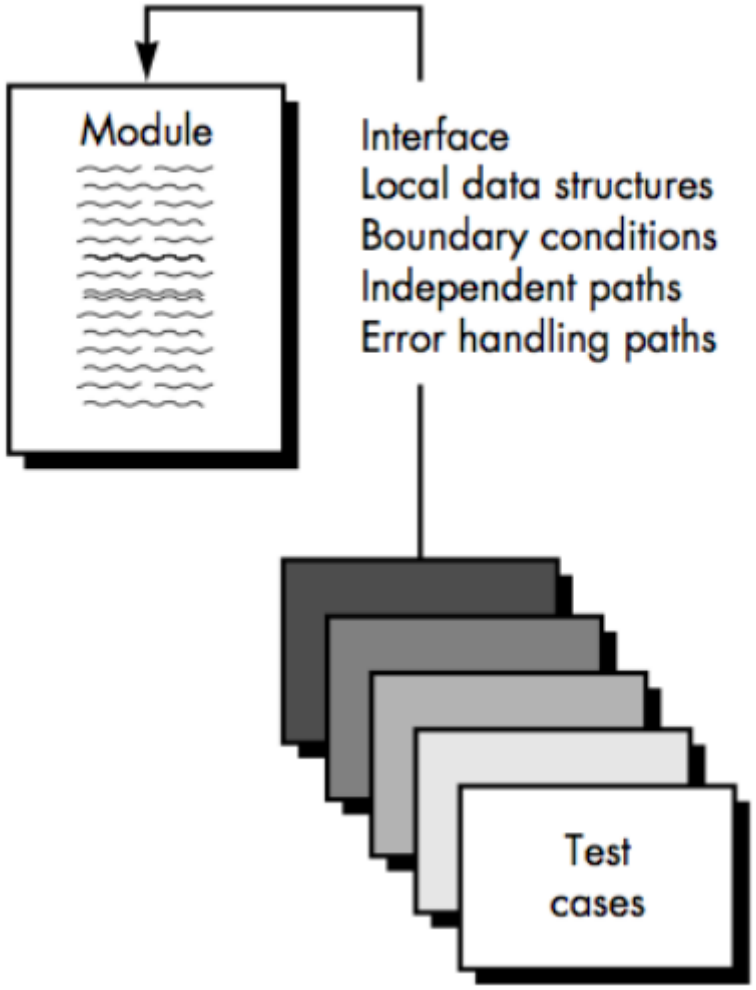


De verificare si validare
Are forma de V
Pe partea stanga am requirements analysis, system design … coding (coboara de sus in jos)
Repr toate fazele dintr-un waterfall in partea stanga
Ideea principala aici e ca pt fiecare faza trebuie sa existe un nivel de testare -> el e derivat din nivelul la care esti
La waterfall clasic, dupa coding te astepti sa fie verificare si validare -> ceea ce face modelul in V : partea de verificare si validare are 2 parti si e integrata la fiecare faza
Are cum sa facem verificare si validare la requirements de ex? -> la mom in care am requirements, pot avea un caz de test de acceptanta -> ce e un caz de test?
E un set de inputuri si outputuri (testele de performanta sunt mai diferite) -> poti sa le definesti, in schimb nu poti sa le rulezi, dar e ok si asa
In fiecare faza -> avem in minte testarea -> fiecare faza produce un nivel de testare la nivelul la care e faza
La requirements -> facem acceptance test
La system design -> facem system test
La nivel arhitectural (pt ca aici avem integrarea componentelor) -> facem testare de integrare
La nivelul module design -> facem testare unitara (proiectam fiecare modul si el trebuie testat unitar)
Pana la modelul in V nu s-a facut nicio testare -> s-a facut cealalta jumatate a testarii -> testarea presupune conceperea cazurilor de test -> se pregateste sistematic -> se merge apoi inapoi in V si vedem care sunt stadiile de testare -> pe partea dreapta vom avea (de jos in sus) prima data testarea unitara -> apoi testarea de integrare -> apoi avem testarea de sistem si abia apoi UAT
Modelul in V nu e un model de dezvoltare neaparat, cat e un model care spune la fiecare faza/moment de timp verificarea si validarea sistemului SW : fac asta in prima faza de coborare (partea stanga) prin a construi cazuri de test (aici nu am nevoie de cod), iar in partea dreapta am testarea efectiva
Vin apoi si iau in ordine inversa cazurile de test ca sa pot sa ma asigur ca am testat cuprinzator/riguros sistemul
Prima chestie e definirea cazurilor de test pe care vreau sa le acopar pt fiecare faza -> apoi trebuie sa aleg datele de test dupa ce gasesc cazurile (ex: nr negative, subunitare etc.)
Tu stii pt datele de intrare de test ce date de iesire trebuie sa obtii -> rulezi testele, vezi ce rez ai obtinut si le compari cu rez la care te asteptai

The software testing process



Unit testing



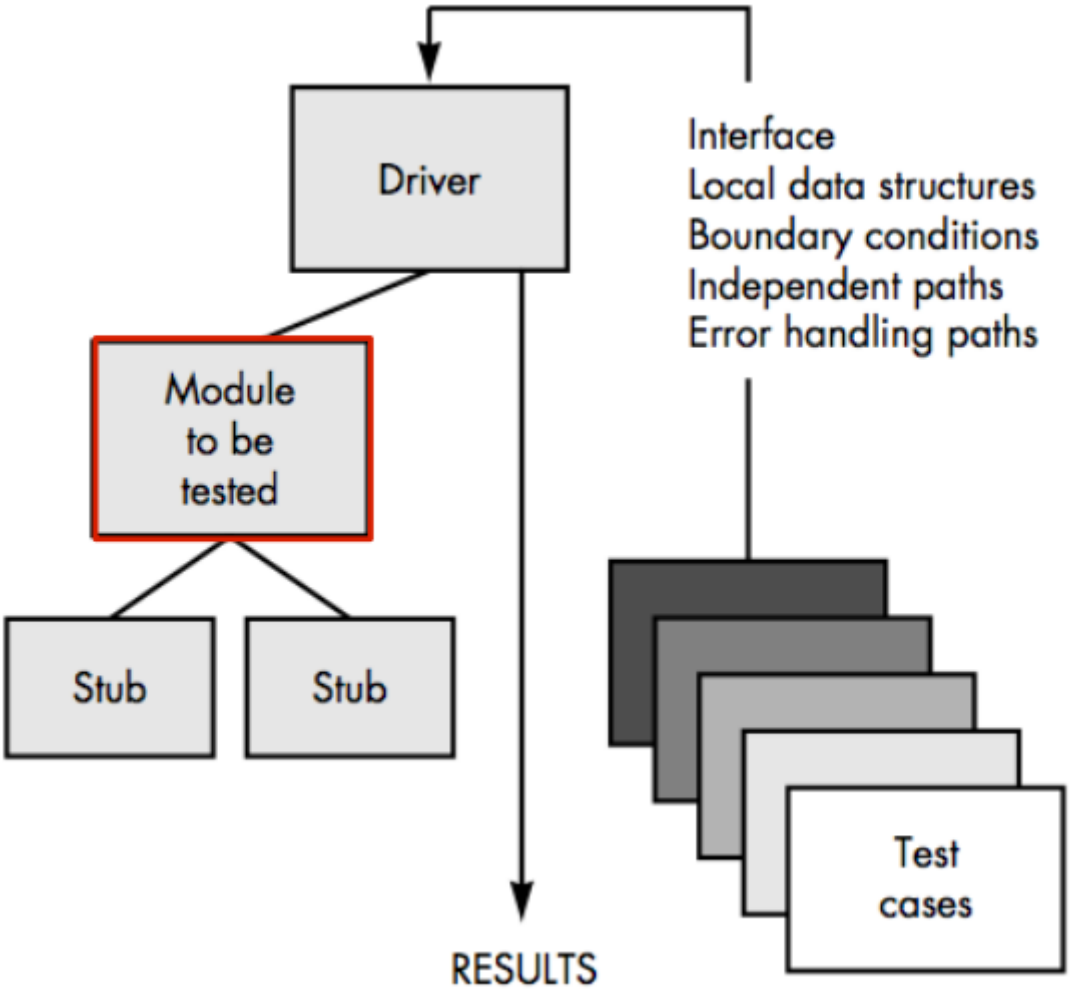
Este o forma de testare low-level -> dar ridica cele mai mari probleme

Targets for Unit Test Cases

- Module interface
  - Ensure that information flows properly into and out of the module
- Local data structures
  - Ensure that data stored temporarily maintains its integrity during all steps in an algorithm execution
- Boundary conditions
  - Ensure that the module operates properly at boundary values established to limit or restrict processing
- Independent paths (basis paths)
  - Paths are exercised to ensure that all statements in a module have been executed at least once
- Error handling paths
  - Ensure that the algorithms respond correctly to specific error conditions

```
Testez fiecare modul in parte, in izolare -> decuplez modulul de toate
celelalte module
Daca sistemul e prost proiectat -> ft greu sa testez unitar
Un moc(stub) e ceva fals -> o pseudodependenta
Izolarea modulelor e cea mai mare problema aici!!!!
O unitate e o clasa in POO
Ce testez?
    o Interfata
    o Structurile de date (datele raman neafectate)
    o Boundary conditions (ne asiguram ca modulul merge ok in valori limita)
    o Caile independente (ne asiguram ca toate path-urile au fost executate
cel putin o data)
    o Caile de exceptie (ne asiguram ca algoritmii trateaza corect
exceptiile)
```

Unit Test Environment



Drivers and Stubs for Unit Testing

- Driver
  - A simple main program that accepts test case data
  - passes such data to the component being tested
  - prints the returned results
- Stubs
  - replace modules that are subordinate to (called by) the tested component
  - uses the module's exact interface
    - may do minimal data manipulation, provides verification of entry, and returns control to the module undergoing testing
- Drivers and stubs both represent overhead
  - must be written but don't constitute part of the installed software product

Driver-ele sunt bune cand testam ceva low level si avem nevoie de legatura
Stub este invers, cand testezi de sus in jos, cand inlocuiesti modulele low level cu un stub

```
Am un modul pe care vreau sa il testez -> acesta are module de care depinde
si atunci le transformam in STUB-URI (repr pseudomodule care dau date
modulului testat ca sa ii asigure functionarea)
Driver-ul e ca un program principal -> ca un main care emuleaza ce e
deasupra modulului pe care vreau sa il testez -> stub-urile inlocuiesc
modulele de dedesubtul unui modul pe care vrem sa il testam -> ca sa scriem
stub-uri si drivere avem costuri in plus -> doar pt testare, ele sunt
necesare
Cand avem new Ceva() -> acel new e hardcodat, nu poate fi inlocuit -> devine
foarte greu de mocuit/de inlocuit cu un stub
Orice referinta a unei subclase poate fi folosita in locul unde se cere sa
avem o referinta a clasei principale -> ajuta ft mult in testare
E ft greu sa mocui/ sa izolezi/sa faci unit testing adevarat pt module care
au multe dependente statice
```

Integration testing

Integration Testing

- Why is unit testing not enough?
  - interfacing causes problems
- Examples of Interfacing Problems:
  - data loss across an interface
  - adverse effects of one module on another
  - individually acceptable imprecisions, may not be globally acceptable
- Two Options:
  - "Big Bang" approach - **NO!**
  - **Incremental Integration**

Urcam un nivel -> vrem sa testam sistemul in ansamblu -> testarea unitara nu e suficienta -> trebuie sa testam un modul impreuna cu alte module

"Big-Bang" Integration Testing

- non-incremental integration

- All components are combined in advance
- The entire program is tested as a whole
- Chaos results
  - Many seemingly-unrelated errors are encountered

- Correction is difficult because isolation of causes is complicated
- Once a set of errors are corrected, more errors occur
  - testing appears to enter an endless loop

-> Scrii tot codul si-l testezi la final. Apar multe erori, care initial nu par a fi corelate, e dificil sa le rezolvi pentru ca te pierzi intre ele. Odata ce corectezi cateva erori, apar altele, pentru ca e greu sa le izolezi

```
Cum o facem? BIG-BANG approach -> nu e o testare de integrare !!!! -> face
parte din categoria asa NU : pui toate modulele impreuna si le dai drumul ->
rulezi niste teste pe tot -> problema e localizarea erorilor : ce a cauzat
problema? -> aceasta testare BIG-BANG nu e buna pt ca nu poti sa izolezi
cauzele, nu se pot localiza exact problemele
```

Incremental Integration Testing

- The program is constructed and tested in small increments

- **Advantages**
  - Errors are easier to isolate and correct
  - Interfaces are more likely to be tested completely
  - A systematic test approach is applied

- Two Strategies
  - Top-down integration
  - Bottom-up integration

Modul logic si normal de a face lucrurile e o integrare incrementala : luam gradual modulele si le integram pe rand, apoi testam

Avantaje :
Erorile sunt mult mai usor de izolat si corectat
Interfetele au mai multe sanse sa fie testate complet
Un mod sistematic de testare este utilizat

Are doua strategii de care vorbim in continuare

Top-Down Integration

- Modules are integrated by moving downward through the control hierarchy, beginning with the main module
- Subordinate modules are incorporated in either a depth-first or breadth-first fashion
  - Depth-First: All modules on a major control path are integrated
  - Breadth-First: All modules directly subordinate at each level are integrated

- **Advantages**
  - verifies major control or decision points **early** in the test process

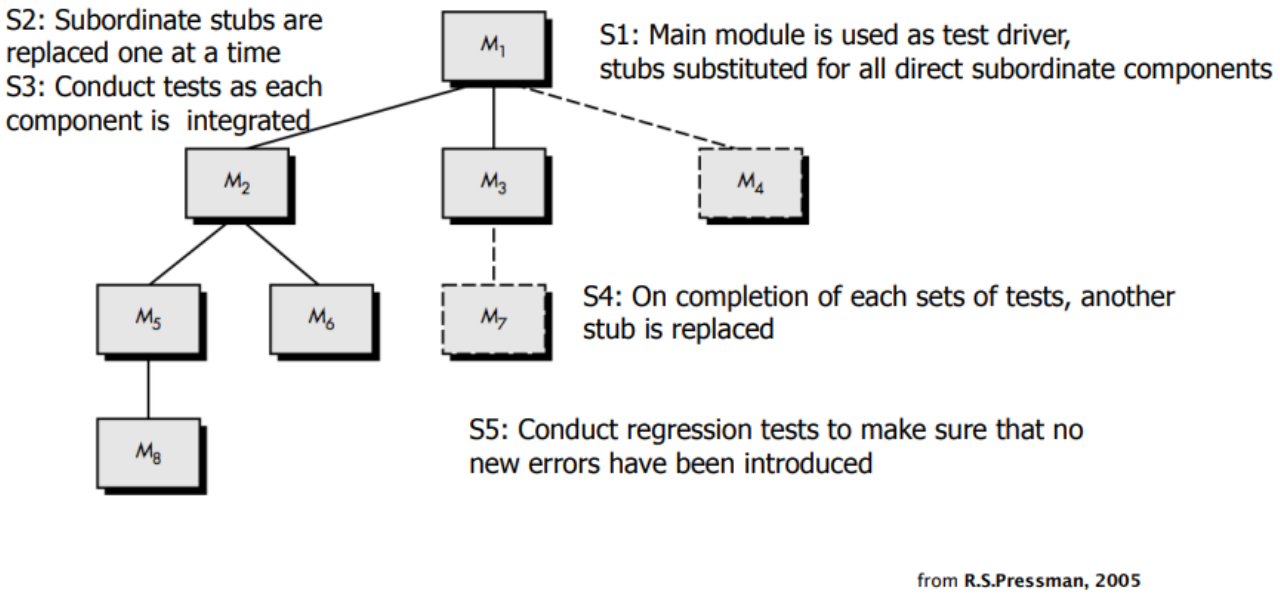
- **Disadvantages**
  - Stubs need to be created for modules that have not been built or tested yet
  - No significant data flow can occur until much later in the integration process
    - because stubs are used to replace lower level modules

Testam de sus in jos (de la main)

Cele mai complexe programe vor fi cele mai testate(si primele de altfel)

E greu sa faci stubs pentru module care inca nu sunt construite

```
▪ Orice aplicatie are un graf de dependente : programul principal -> e un
graf aciclic orientat -> dar daca e ciclic? Tot ce ce ciclic e intr-un
singul modul
▪ Sus inseamna modulul pe care nu-l apeleaza nimeni
▪ De ex, am integrat modulul m1 impreuna cu modulul m2, pe m3 si m4 nu vreau
inca sa le integrez -> folosesc stub-uri ca sa le substitui
▪ Avantaje : pornind de sus, eu testez ft bine logica de nivel inalt ->
punctele de control se testeaza repede, cu prioritate si des -> pe masura ce
testez si urmatoarele, cele de sus se testeaza cel mai mult pt ca se
testeaza si impreuna cu urmatoarele
▪ Dezavantaje : trebuie sa creez stub-uri si datele cu care se lucreaza nu
sunt date reale, sunt stub-uri
▪ Integrarea se poate face si in latime(am acoperit un nivel si pot sa cobor
gradual) si in adancime (cel putin pe o ramura exersez si data flow, nu doar
control flow)
```



Regression test -> testare suplimentara

Bottom-up Integration

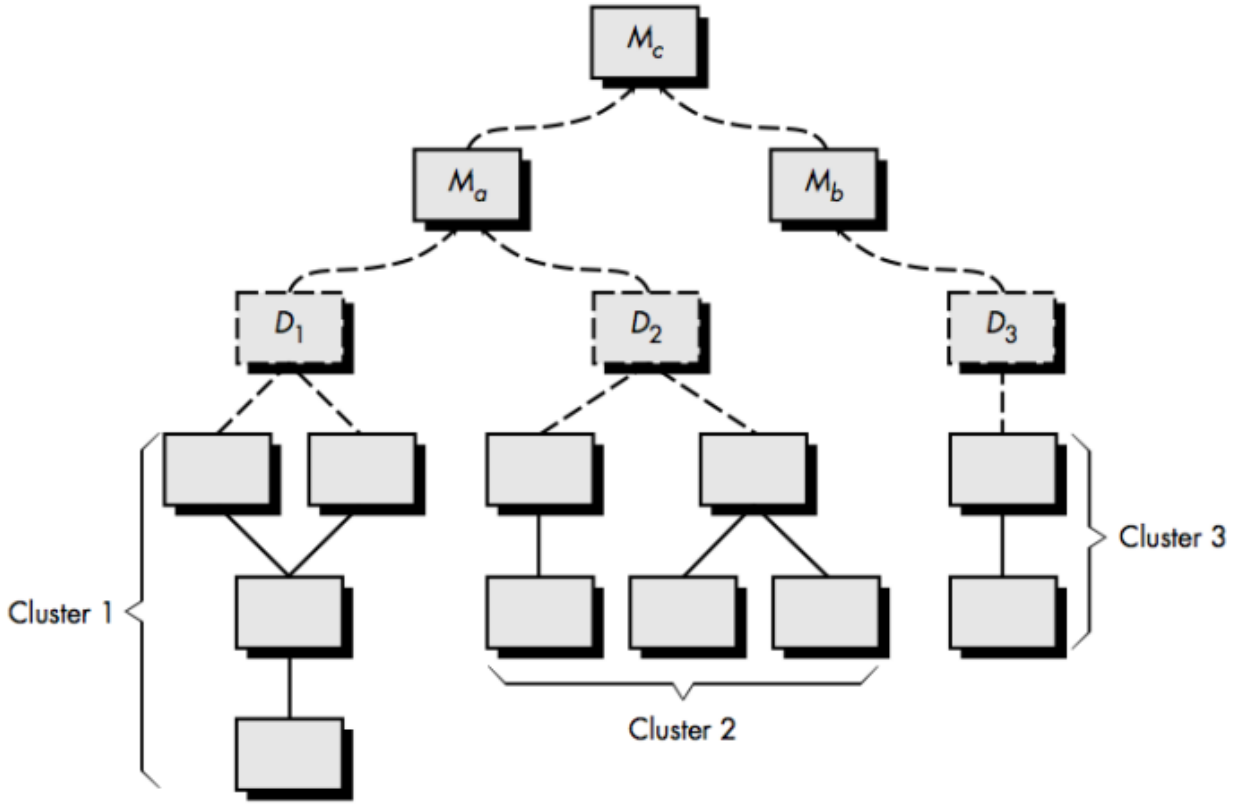
- Integration and testing starts with the most atomic modules in the control hierarchy

- **Advantages**
  - verifies low-level data processing early in the testing process
  - No need for stubs

- **Disadvantages**
  - Driver modules needed
    - to test the lower-level modules
    - this code is later discarded or expanded into a full-featured version
  - Drivers are inherently incomplete
    - do not contain the complete algorithms that will eventually use the services of the lower-level modules
    - testing may be incomplete or more testing may be needed later when the upper level modules are available

Incepem de jos in sus

```
▪ Module care nu mai depind de nimeni
▪ Avantaje : aici nu am nevoie de stub-uri, pt ca pornesc de la frunze
▪ Dezavantaje : am nevoie de drivere (contin o logica de utilizare) ->
driverele sunt ca un fake main module -> dezavantajul e ca business logicul
cu care testezi nu e cel real, ci e unul dummy, fake -> pe de alta parte,
data flow-ul e ft bine testat + Driverele pot fi incomplete, la fel si
testarea si va fi nevoie de o interventie ulterioara cu mai multe teste,
cand module superioare vor fi disponibile
▪ Ca mod de integrare se merge clustere, nu se integreaza modul cu modul,
practic pornesti de la mai multe frunze si faci un grup de frunze si pe ala
il integrezi -> apoi deasupra lor pui un driver (un main fake) -> apoi in
locul driverului se pune modulul real -> repet pana ajung in varf
```



Regression Testing

- Each new addition or change to baselined software may cause problems with functions that previously worked flawlessly
- Regression testing re-executes a subset of tests that have already been conducted
  - Ensures that changes have not propagated unintended side effects
    - changes do not introduce unintended behavior or additional errors
  - May be done manually or through the use of automated capture/playback tools

- Regression test contains 3 classes of test cases
  - A representative sample of tests that will exercise all software functions
  - Additional tests on software functions likely to be affected by the change
  - Tests that focus on the actual software components that have been changed

Reexecuta teste pe tot programul, nu doar pe ce a fost adaugat nou

Poate fi facut manual sau automat

Are 3 clase de test case-uri

1. pe toate functiile software
2. pe functiile care puteau fi afectate de schimbari
3. pe componenta care a fost schimbata

```
Am integrat, testat, totul merge -> NU! -> am tot sistemul testat cu testare
de integrare , dar in continuare avem cazuri de test -> testez de ex o
bucata/un slice de cod, dar nu este suficient -> apare aici testarea de
regresie -> trebuie sa ne asiguram ca sistemul nostru nu a regresat (de ex,
daca am introdus noi feature-uri, sa nu fi introdus niste bug-uri
neasteptate
Cum se face testarea de regresie : scriu teste noi pt integrarea noului
feature, dar pe langa asta mai rulez niste teste care au fost deja rulate
(un subset de teste, nu toate testele care au fost rulate)
O testare de regresie inseamna ca sa re-executi un subset de teste care au
fost deja rulate -> avem teste care acopera o zona mai larga, cumva si in
exteriorul imediat apropiat al modulelor testate
```

Smoke Testing

- Taken from the world of hardware
  - Power is applied and a technician checks for sparks, smoke, or other dramatic signs of fundamental failure
- Designed as a pacing mechanism for time-critical projects
  - Allows the software team to assess its project on a frequent basis
- Includes the following activities
  - The software is compiled and linked into a build
  - A series of breadth tests is designed to expose errors that will keep the build from properly performing its function
    - The goal is to uncover "show stopper" errors that have the highest likelihood of throwing the software project behind schedule
  - Build is integrated with other builds and the entire product is **smoke tested daily**
    - Daily testing gives managers and practitioners a realistic assessment of the progress of the integration testing
  - After a smoke test is completed, detailed test scripts are executed

E o testare superficiala, cam deloc -> ruleaza un subset minimal de teste, dar care acopera principalele chestii care ar putea sa dea fail ft naspa

Scopul ei e sa gaseasca erori de suprafata, erori care au sansa cea mai mare sa apara (show stopper errors -> legate de infant mortality)

Smoke testing trebuie facut daily

Benefits of Smoke Testing

- Integration risk is minimized
  - Daily testing uncovers incompatibilities and show-stoppers early in the testing process, thereby reducing schedule impact
- Error diagnosis and correction are simplified
  - Smoke testing will probably uncover errors in the newest components that were integrated
- Progress is easier to assess
  - As integration testing progresses, more software has been integrated and more has been demonstrated to work
  - Managers get a good indication that progress is being made

Daca faci smoke testing, vei descoperi la timp erorile si nu vei da delay la schedule
Diagnoza erorilor si corectiilor este simplificata
Progresul e usor de trackuit

```
Dupa ce am terminat cu integrarea -> vedem daca trece de smoke test ->
double check -> ma asigur ca dupa ce am facut un build, am rulat toate
testele din build(nu sunt toate testele din sistem) -> ma asigur ca build-ul
ala compileaza -> dupa ce trece de smoke test pot sa il pun undeva de unde
poate fi utilizat
```

Test Case Design

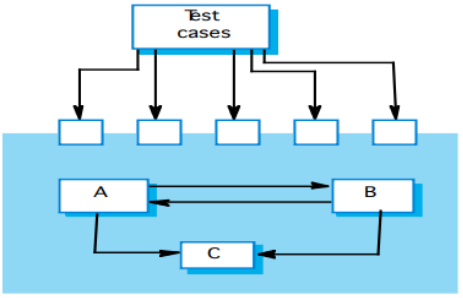
What is a “Good” Test?

- A good test has a high probability of finding an error
- A good test is not redundant.
- A good test should be “best of breed”
  - reveals more than one error... an entire class of errors
- A good test should be neither too simple nor too complex
  - if too simple, might not reveal anything
  - if too complicated danger of side effects
  - Tests should be run individually

Cum proiectez cazuri de test? Ce e un test bun? -> Are sanse bune sa gaseasca o eroare si un test e bun daca nu e redundant

Testing guidelines

Testing guidelines

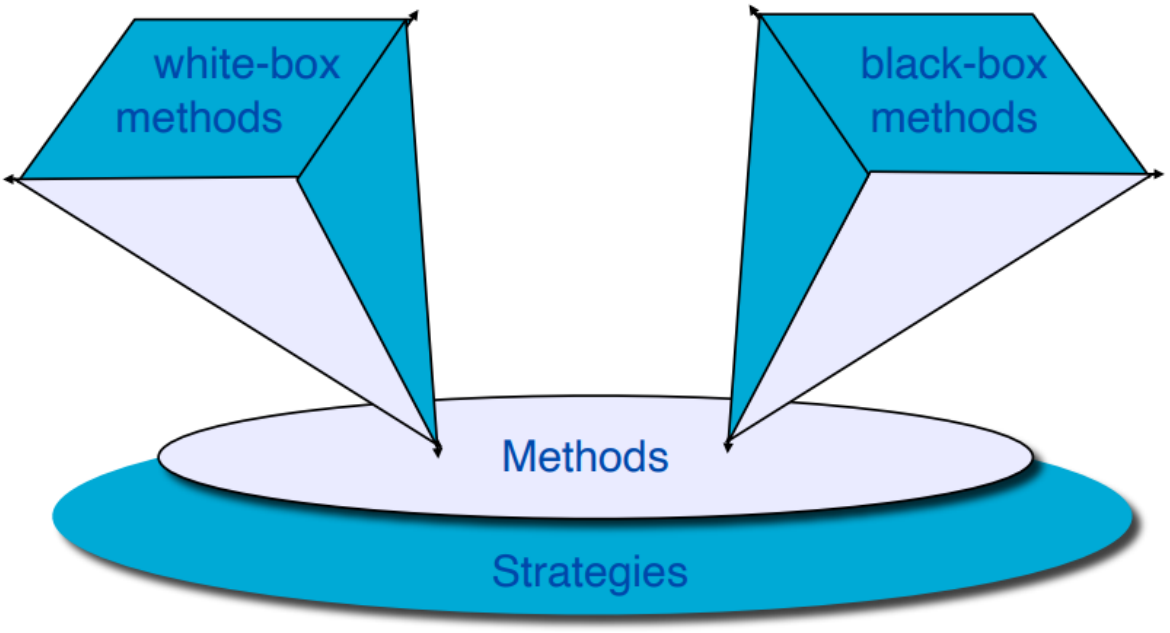


- Design tests so that parameters to a called procedure are at the extreme ends of their ranges.
- Always test pointer parameters with null pointers.

Design tests which cause the component to fail.

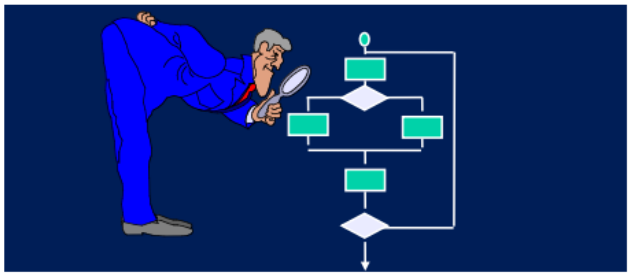
Mereu incercam sa ne ducem parametrii in cazuri limita
Pointerii trebuie sa fie testati si cu valoarea NULL
Scopul e sa pice testele :)

Software Testing



La black box vorbim de input/output, iar la white box de caile din program
Strategie -> unit/integration

White-Box Testing



... our goal is to ensure that all statements and conditions have been executed at least once ...

from **R.S.Pressman, 2005**

- The starting point for path testing is a program flow graph
  - nodes = program decisions
  - arcs = flow of control.
  - Statements with conditions are therefore nodes in the flow graph.

Aici stim deja codul si ne scriem testele astfel incat sa acoperim toate ramurile posibile

Why Cover?

- Logic errors and incorrect assumptions are inversely proportional to a path's execution probability
  - more errors in rarely executed paths
- We often **believe** that a path is not likely to be executed
  - reality is often counter-intuitive
- typographical errors are random
  - likely that untested paths will contain some

Path-urile mai putin traversate sunt de obicei cele care genereaza cele mai mlte erori

```
o Vrem sa ne asiguram ca am parcurs toate ramurile din program -> code
coverage : am trecut cu cate un test prin fiecare bloc
o Coverage ajuta cu un singur lucru : sa gasim erori stupide care se pot
strecura oriunde in cod
```

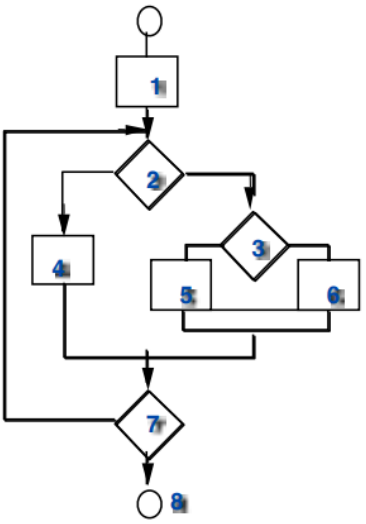
Cyclomatic Complexity [McCabe, 1976]

- Definition:

Let $G(N, E)$ be a control flow graph (CFG). Cyclomatic complexity is defined as

$$v(G) = e - n + 2$$

where $e = |E|$ and $n = |N|$.



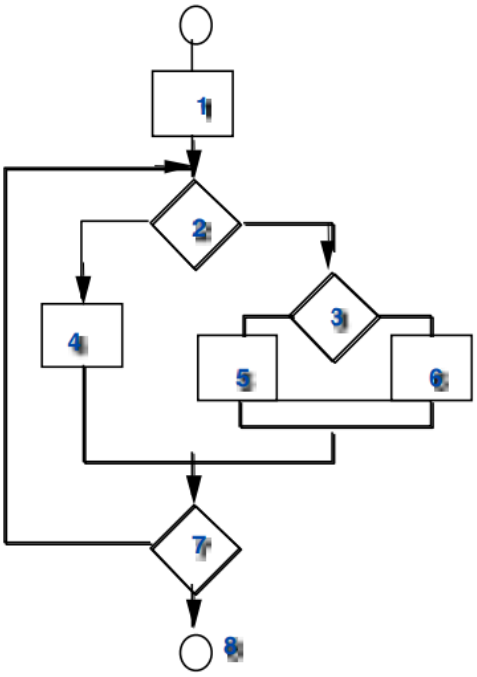
- Interpretation:

more cyclomatic complexity = more branching =
=more testing needed

Cu cat ai mai multe ramuri, cu atat ai nevoie de mai multe teste

Basis Path Testing

Basis Path Testing



First, we compute the cyclomatic complexity:

number of simple decisions + 1

or

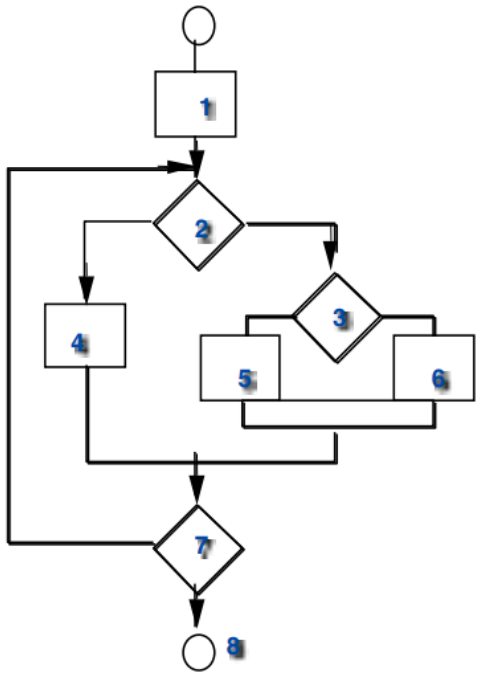
number of enclosed areas + 1

In this case, V(G) = 4

```
o Aici avem complexitatea ciclomatica : repr nr de cai independente pe care
le are un program (nr de suprafete inchise) -> nr de decizii simple + 1 ->
tinem cont ca o decizie poate fi si compusa ( ex: a sau b, a si b etc.)
o Daca avem de ex o instructiune if-else -> complexitatea ciclomatica e 2
,daca avem assign -> complexitatea e 1
```

```
o In for si while ai o decizie de facut, practic conditia de mers mai
departe in loop
```

Basis Path Testing



Next, we derive the independent paths:

Since V(G) = 4, there are four paths

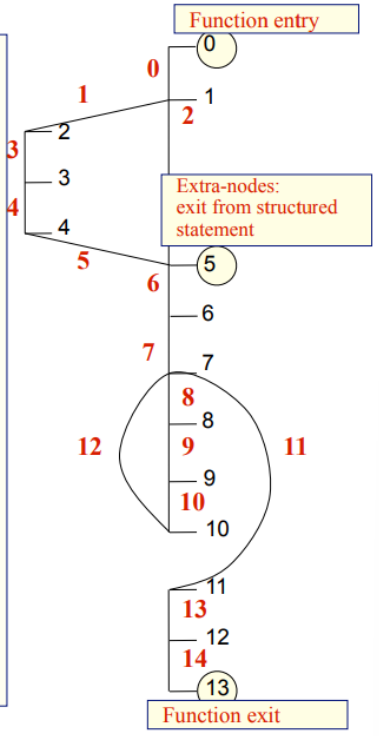
Path 1: 1,2,3,6,7,8
Path 2: 1,2,3,5,7,8
Path 3: 1,2,4,7,8
Path 4: 1,2,4,7,2,4,...7,8

Finally, we derive test cases to exercise these paths.

Cyclomatic Complexity for Euclid algorithm

Cyclomatic Complexity for Euclid algorithm

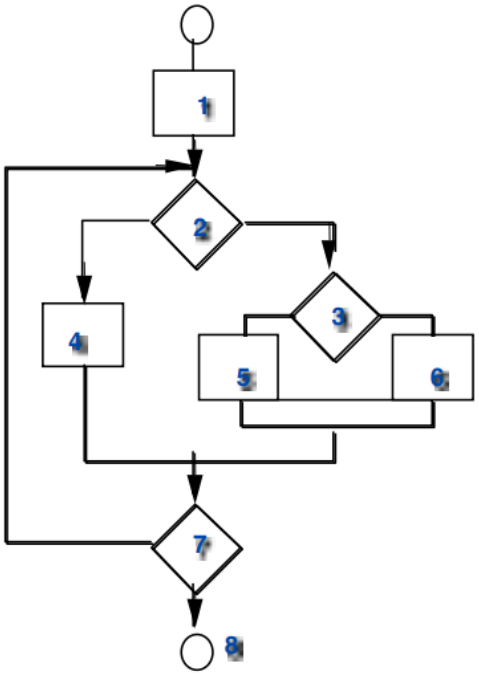
```
int euclid(int m, int n)
{                                        /*0*/
    int r;
    if(n > m)                           /*1*/
    {
        r = m; m = n; n = r;            /*2,3,4:*/
    }                                   /*5*/
    r = m % n;                          /*6*/
    while(r != 0)                       /*7*/ {
        m = n; n = r; r = m % n;        /*8,9,10:*/
    }                                   /*11*/
    return r;                           /*12*/
}                                       /*13*/
```



$v(G) = 15 - 14 + 2 = 3$

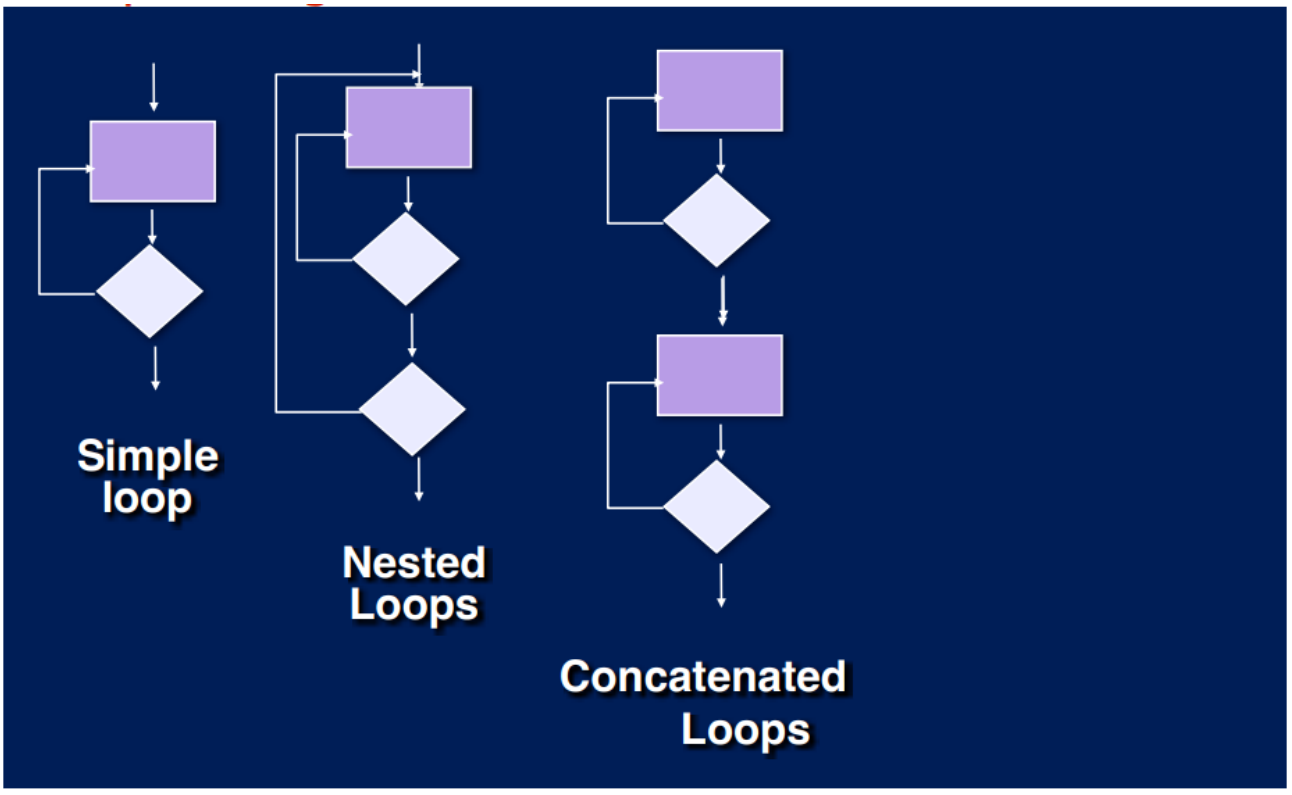
Dr. Radu Marinescu X

Remarks on Basis Path Testing



- You don't need a flow chart, but the picture will help when tracing the program paths
- Count each simple logical test!
  - Compound tests count as >= 2
- Basis Path Testing should be applied to critical modules

Loop Testing



Loop Testing: Simple Loops

- Minimum Conditions to Test:
  1. Skip the loop entirely
  2. Only one pass through the loop
  3. Two passes through the loop
  4. N-1 and N passes through the loop

where N is the maximum number of allowable passes

Cum testam buclele? Ex : un for -> mai multe cazuri :

- Simple loop :
- 1. Dam skip la intregul loop
- 2. facem o singura trecere prin loop
- 3. 2 treceri prin loop
- 4. N-1 si N treceri prin loop -> N e nr maxim de pasi pe care ii putem face

Loop Testing: Nested Loops

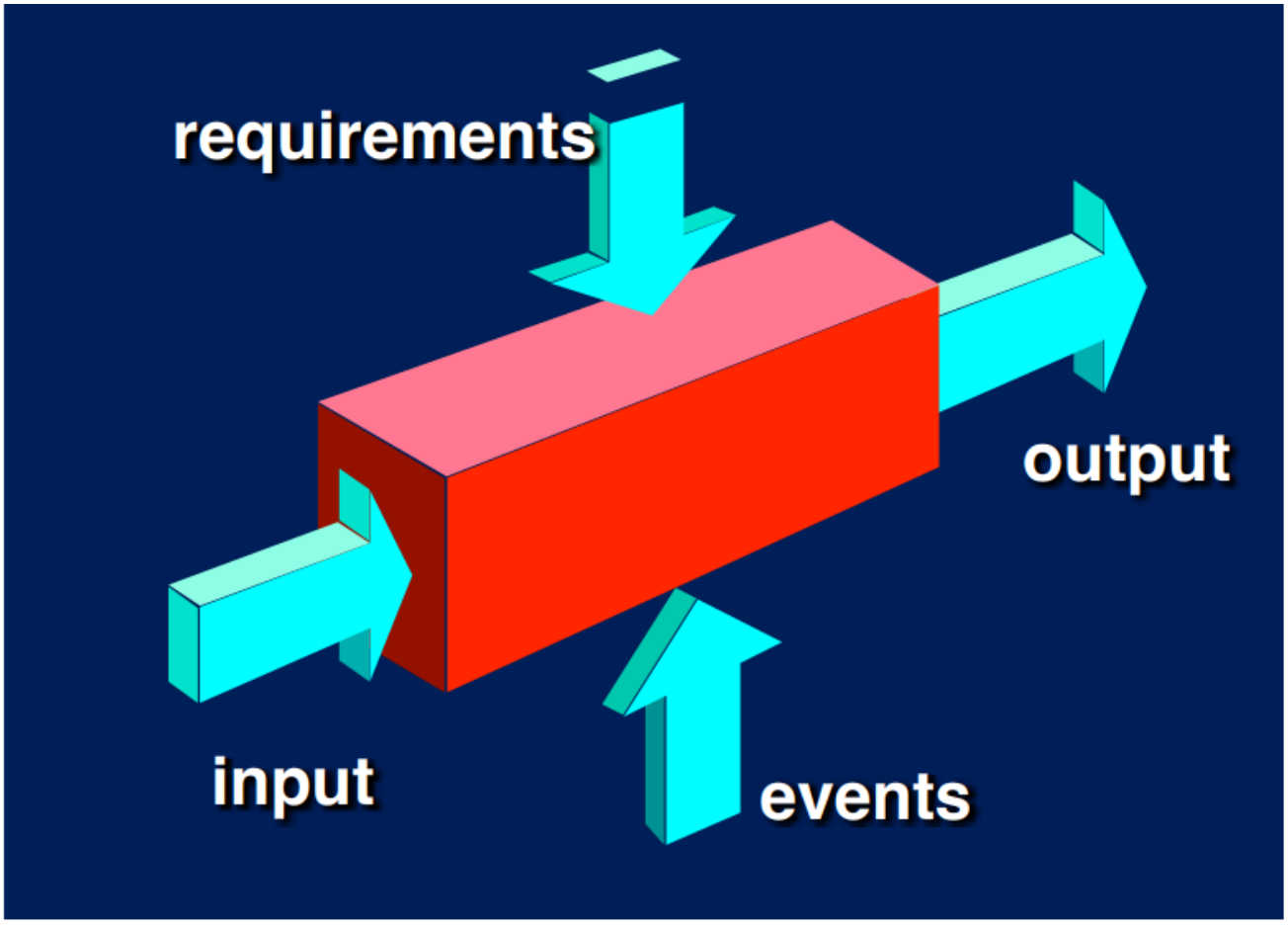
- **Nested Loops**
  1. Start at the innermost loop. Set all outer loops to their minimum iteration parameter values
  2. apply **simple loop testing rules** to the innermost loop, while holding the outer loops at their minimum values
  3. Move out one loop and set it up as in step 2, holding all outer loops at minimum values, and all inner loops at typical values.
  4. Continue step 3 until the outermost loop has been tested

- **Concatenated Loops**
  6. If the loops are independent of one another, treat each as a simple loop
  7. Else treat as a nested loop (e.g. the final loop counter value of loop 1 is used to initialize loop 2

- Nested loops (ciclu compus : for in for) – mergem din interior in exterior
  - 1. Testarea se incepe de la ciclul cel mai interior (setam parametrii la minim din cele mai exterioare loop-uri)
  - 2. Aplic cel mai simple reguli de testare pe cel mai interior ciclu, in timp ce tinem ciclurile in exterior la valorile lor minime
  - 3. Fixez ciclul interior la o valoare tipica si ma concentrez asupra ciclului exterior : il rulez cu 0 intrari, 1 intrare etc.
  - 4. Repet pasul 3 pana cand cel mai exterior ciclu a fost testat

```
• Daca avem cicluri concatenate -> ai un for si apoi alt for -> un ciclu
concated repr un ciclu simplu, unul dupa altul, un for dupa for -> le tratam
ca un for simplu, sau ca nested daca valoarea finala counter loop 1 =
valoarea initiala counter loop 2
```

Black-Box Testing



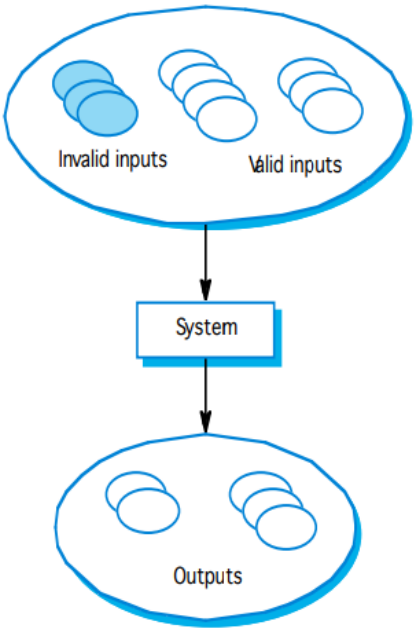
Nu stim codul, nu ne intereseaza code coverage
Ne intereseaza doar input si output

Partition testing

- Input data and output results often fall into different classes where all members of a class are related.
- Each of these classes is an equivalence partition
  - i.e a domain where the program behaves in an equivalent way for each class member.
- Test cases should be chosen from each partition.

```
o Am niste cerinte/evenimente -> in principal setez datele de intrare,
gasesc valorile lor pe care vreau sa le dau -> vad rezultatele si apoi
compar rez obtinute cu cele corecte
```

Equivalence partitioning



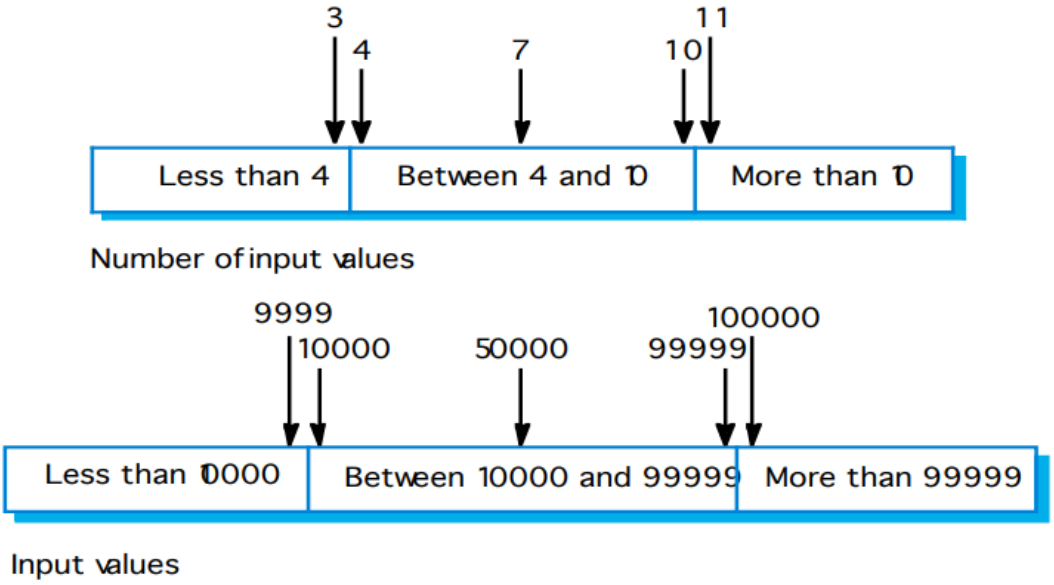
- Identify partitions from program specs or user documentation
- For each partition choose test cases:
  - **boundaries**
  - **mid-point**

```
o Am un vector de valori -> cate valori trebuie sa dau? Trebuie sa
partitionez valorile pe care le am
```

-> Luam valorile limita (boundary) si valorile de mijloc

Equivalence partitions

- Example: program accept **4 to 10 inputs** that are **five-digit integers > 10.000**




```
o Partitiile echivalente -> modulul meu accepta un vector de valori -> intre
0 si 100 valori de ex -> cum in testare am un nr limitat de incercari ->
impart in niste grupe si apoi aleg din fiecare grupa -> impart toate
inputurile in niste gramajoare -> verificam inputurile valide si invalide
▪ Impartim inputurile valide in niste gramajoare -> consideram ca fiecare
valoare din grup e echivalenta cu toate celelalte -> rezervam cateva teste
si pt cazurile invalide
▪ Pt fiecare partitie putem sa alegem 1 test sau 3 teste -> ex : am setul de
valori 100 -> am 3 cazuri : putine valori, medii si multe valori -> spun de
ex ca intre 1 si 5 am putine valori , intre 5 si 30 sunt medii si peste 30
sunt multe
▪ Ex : avem un program care accepta in 4 si 10 inputuri -> vector intre 4 si
10 valori -> fiecare valoare repr un nr de 5 digits si trebuie sa fie
valoare mai mare de 10.000 : aleg 3 cazuri de test pt nr de valori : daca am
mai putin de 4 valori, daca am intre 4 si 10 valori si daca am peste 10
valori -> aleg 3 cazuri de teste pentru nr de 5 digits : pot sa am nr sub
10.000 (sub 5 digits -> 4 digits), nr cuprinse intre 10.000 si 99.999 si pot
sa am nr peste 99.999 (peste 5 digits -> 100.000 -> 6 digits)
▪ Trebuie mereu sa ma uit care ar fi invalid inputs si valid inputs -> ma
uit mereu la margini si mijloc -> testarea nu se face exhaustiv, ci se face
“good enough”
```

Guidelines for testing sequences (collections)

- Test software with sequences which have only **a single value**.
- Test with sequences of **zero length**.
- Use **sequences of different sizes** in different tests.
- Derive tests so that the **first**, **middle** and **last elements** of the sequence are accessed.

o Testez colectii care n-au nicio valoare, apoi o valoare etc.

o Incerc colectii/secvente cu diferite dimensiuni